Genome Alchemy v6.1

Documented Taxonomy-Aware Parallel Framework

This PDF documents the fully commented **genome_alchemy_v6_taxa_parallel_documented.py** script. It is designed for GitHub, collaboration, and future extension. The documentation emphasizes architecture, taxonomy handling, controls, parallelism, outputs, and practical usage.

Primary goals	Falsifiability, reproducibility, scalability, taxonomy awareness
Primary input modes	NCBI RefSeq/GenBank accessions and local FASTA files
Primary compute model	Window-level parallel feature extraction with ProcessPoolExecutor
Primary outputs	CSV summaries, JSON reports, optional figures, manifest, and logs

1. Pipeline architecture

The pipeline below shows how Genome Alchemy moves from raw genomes to windowed features, control comparisons, and final reports. The workflow is intentionally explicit so every strong claim can be traced to concrete stages.

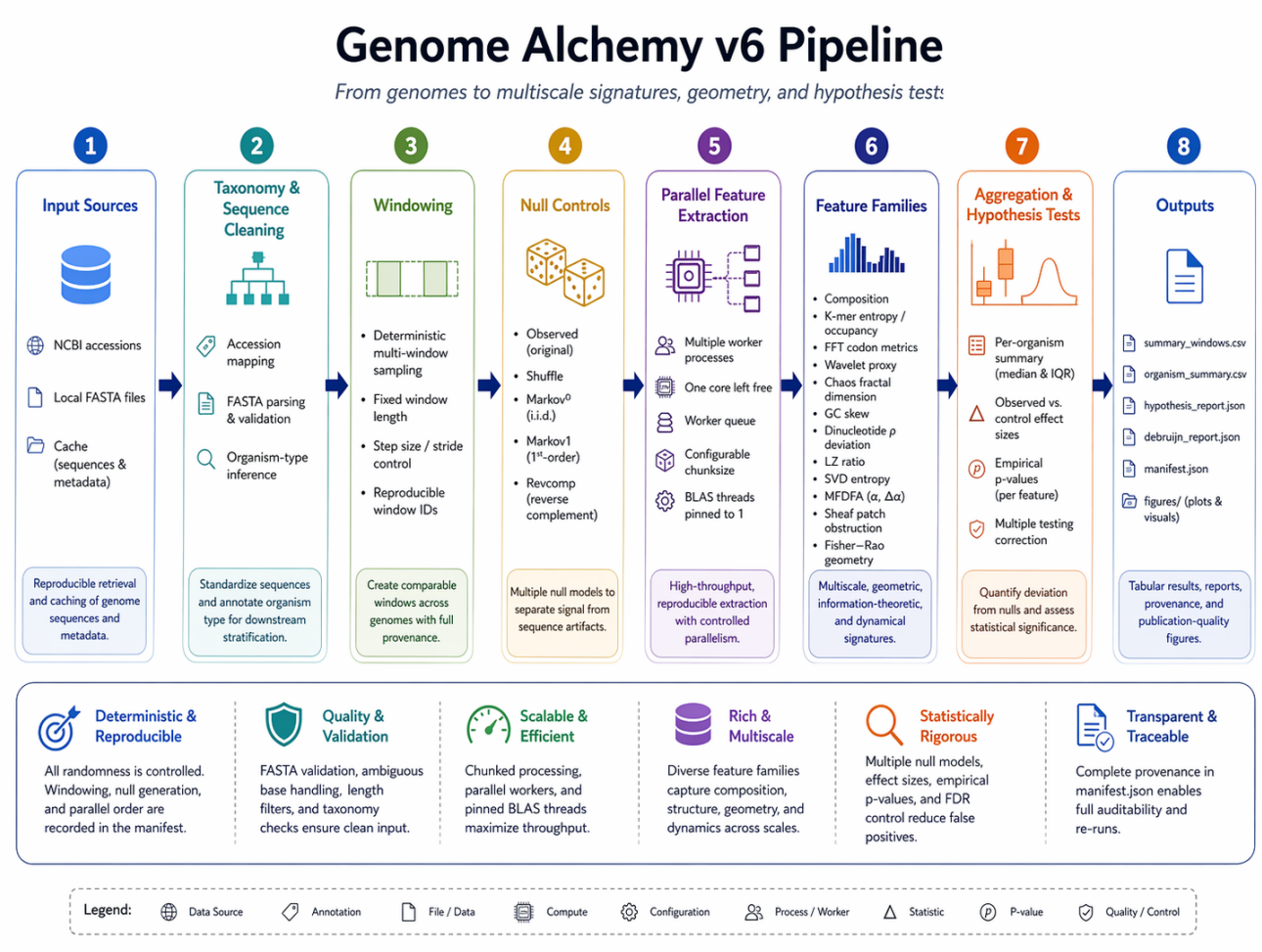

Figure 1. High-level pipeline overview generated as a documentation diagram.

The most important conceptual move is the inclusion of **null controls** as first-class objects. Instead of only describing observed sequences, the framework creates matched surrogate windows and asks whether the observed data differ meaningfully from those baselines.

2. Dataset tiers and taxonomy

A major redesign in this version is explicit handling of more organism types. Humans are no longer treated as bacteria, and the broad eukaryotic panel helps validate whether patterns generalize across more representative lineages.

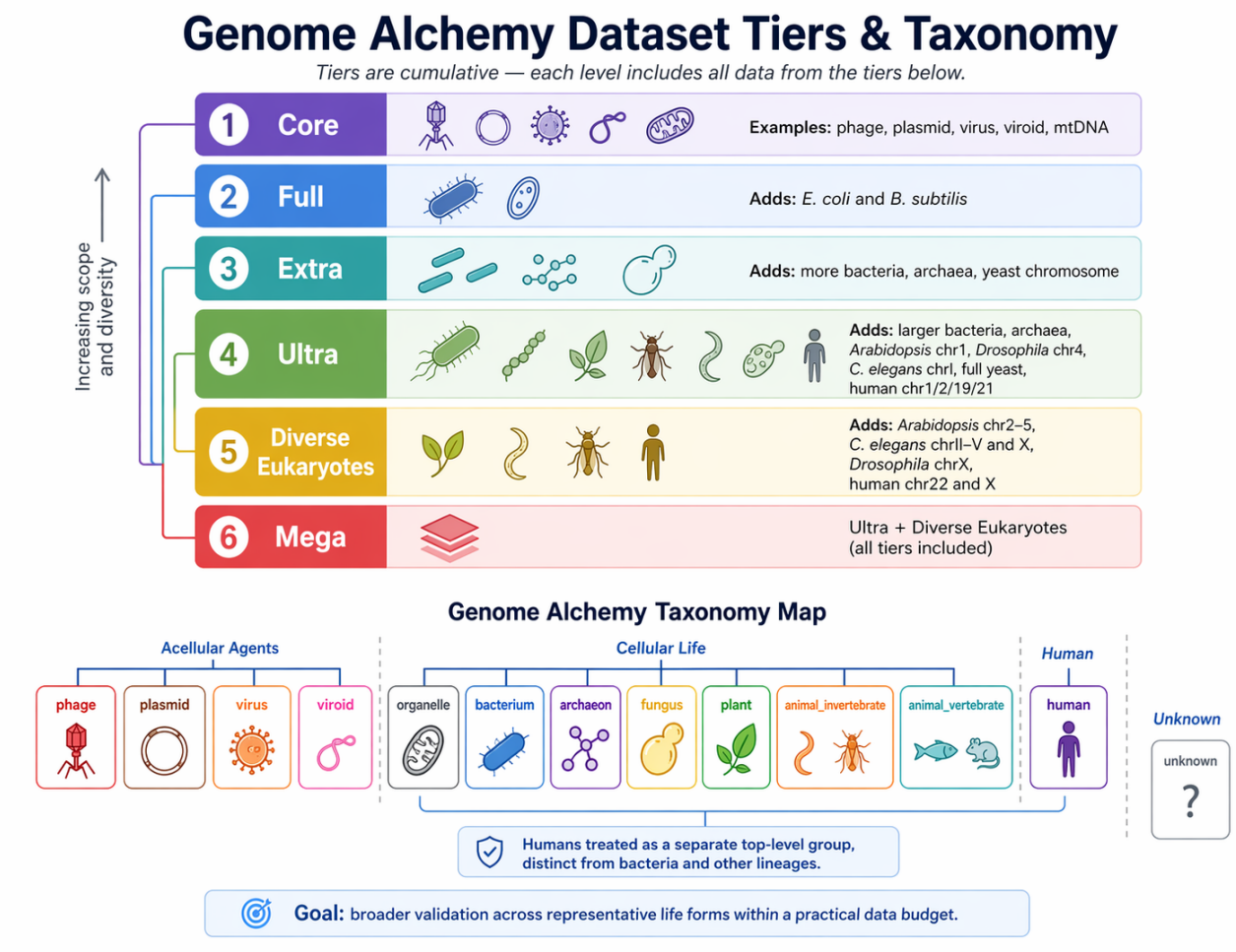

Figure 2. Cumulative dataset tiers and taxonomy map.

3. Command-line interface

Flag	Meaning
--full / --extra / --ultra / --diverse-eukaryotes / --mega	Cumulative accession tiers. --mega enables the broadest panel.
--window	Window length in base pairs. Default 200,000.
--windows-per-genome	Number of deterministic observed windows sampled from each genome.
--controls	Comma-separated list of surrogate models: shuffle, markov0, markov1, revcomp.
--workers	Process count for feature extraction. Use 3 on a 4-core machine to keep one core free.
--chunksize	Task batch size submitted to workers.
--no-plots	Skip figure generation for faster runs.
--no-debruijn	Skip full-sequence de Bruijn diagnostics.
--fasta	Add one or more local FASTA files.

Example run (broad but practical):

```
python genome_alchemy_v6_taxa_parallel_documented.py --mega --email you@example.com \
  --out results_mega \
  --window 200000 \
  --windows-per-genome 4 \
  --controls shuffle,markov1,revcomp \
  --workers 3 \
  --chunksize 4 \
  --no-plots
```

4. Feature panel

- **Composition**: base counts and GC percentage.
- **k-mer spectrum**: entropy and occupancy of k-mer space.
- **FFT codon metrics**: frequency-domain detection of period-3 structure.
- **Wavelet proxy**: lightweight scale-aware codon periodicity summary.
- **Chaos fractal dimension**: approximate box-counting behavior of a sequence walk.
- **GC skew**: strand asymmetry across bins.
- **Dinucleotide rho deviation**: deviation from mononucleotide-expected dinucleotide usage.
- **LZ / zlib ratio**: compression-based complexity proxy.
- **SVD entropy ratio**: structure estimate from a matrix embedding.
- **MFDFA proxy**: fast summary of multifractal-like behavior.
- **Sheaf patch obstruction**: patchwise inconsistency proxy.
- **Fisher–Rao geometry**: distributional distance metrics in k-mer space.

5. Parallelism and performance

The script uses **ProcessPoolExecutor** to distribute windows across worker processes. This is preferable to threads for CPU-heavy scientific Python workloads. To avoid oversubscription, the script pins BLAS/OpenMP thread counts to 1 inside each worker process.

Machine	Suggested workers	Notes
2 cores	1-2	Use 1 if you want a very responsive desktop during the run.
4 cores	3-4	3 is a good balance; 4 maximizes throughput.
8+ cores	cores-1 or cores	Raise chunksize if worker scheduling overhead becomes visible.

On a 4-core desktop CPU, `--workers 3` is usually the sweet spot if you want to leave one core available for other work while the job runs.

6. Outputs and reproducibility

File	Purpose
manifest.json	Records the configuration and all loaded sequences with lengths and hashes.
summary_windows.csv	One row per window/control with extracted features.
organism_summary.csv	Observed-window summary table grouped by organism.
hypothesis_report.json	Observed-vs-control effect summaries.
debruijn_report.json	Optional graph diagnostics for full sequences.
figures/*.png	Plots if figure generation is enabled.
run.log	Full textual log for the run.

Reproducibility is strengthened by stable hashing, deterministic window sampling, persistent cache files, and a manifest that stores enough metadata to reconstruct the run later.

7. Extending the framework

1. Add new accessions to the tier dictionaries near the top of the script.
2. Add new taxonomy aliases to `TAXONOMY_ALIASES` and update `infer_organism_type` if necessary.
3. Implement a new feature function and register it inside `compute_features`.
4. Add a new null model in `make_controls` when stronger baselines are needed.
5. Create additional summary visualizations in `make_figures`.

8. Deliverables created for this documentation pass

- **genome_alchemy_v6_taxa_parallel_documented.py**: fully documented source code.
- **Genome_Alchemy_v6_Documentation.md**: Markdown documentation for repository use.
- **Genome_Alchemy_v6_Documentation.pdf**: this PDF.
- **genome_alchemy_v6_pipeline_overview.png**: pipeline diagram.
- **genome_alchemy_dataset_tiers_and_taxonomy.png**: taxonomy/tier diagram.

End of documentation.